Quantitative Trading with Python: A Top-Down Approach

Jack Forden

March 2026

Contents

1 Physics Before Formulas **1**

1.1 What Is the Market? 1

1.1.1 The Market's Structure 2

1.1.2 The Digital Revolution and the NASDAQ Era 5

1.1.3 The Market as a Real-Time Decision Field 5

1.1.4 Primary vs Secondary Markets 6

1.1.5 Core Market Participants 6

1.1.6 Dealers and Market Making 6

1.1.7 Brokers and Order Routing 6

1.1.8 Financial Intermediaries 7

1.2 System Lens 7

2 From Intuition to Measurement **8**

2.1 What We Measure 8

3 Machine Learning **9**

3.1 Principles of Machine Learning 10

3.2 Supervised Learning 13

3.3 Random Forest 15

Chapter 1

Physics Before Formulas

Today's global financial market is arguably one of the largest engineered systems in existence: millions of instruments, fragmented venues, market makers, brokers, clearing systems, data centers, scripts, and autonomous agents interacting in nanoseconds. The New York Stock Exchange, which started when 24 brokers signed the Buttonwood Agreement in 1792, has grown into a global network of exchanges and trading venues that operate around the clock. In the modern era, the conception of a "market" has expanded to include not only traditional stock exchanges but also electronic trading platforms, dark pools, decentralized finance (DeFi) networks, prediction markets.

When viewing the market as an independent patchwork of venues, participants, and instruments, it is easy to get lost in the details. The goal of this chapter is the opposite: to abstract the market into a system that can be reasoned about, and then use that abstraction to guide the design of trading strategies that are robust by construction, not by accident.

This chapter provides a broad overview to see the forest through the trees: terminology first, then market edge (participants and workflows), market core (venues, execution, and data flow), and finally the intelligence layer (models, agents, risk controls, and failure modes).

1.1 What Is the Market?

When most people hear "the market," they imagine a building, a bell, or a chart on a screen. That picture is not wrong, but it is incomplete enough to hurt you if you trade from it. The market is a network of interacting agents and venues that collectively determine prices and allocate capital, each participant operating under their own objectives, constraints, and information.

To start, we will define the market from a high level perspective, focusing on its structure, participants, and the flow of information and capital. The second half of the chapter will focus on the market from a system engineering perspective, where we explore the market as a complex adaptive system that can be modeled, analyzed, and optimized.

1.1.1 The Market's Structure

The New York Stock Exchange traces back to the Buttonwood Agreement, signed by 24 brokers in New York on May 17, 1792. In modern terms, Buttonwood functioned like a protocol: it defined who could trade with whom and standardized commissions. The point was trust and control, not speed. By requiring a broker for every trade, the agreement ensured that every transaction passed through a verified member who was personally responsible for the deal's success.

Prior to the Buttonwood Agreement, trading was far more informal, with brokers negotiating deals directly with each other in coffee houses and other ad hoc venues. The lack of structure often led to disputes, inconsistent pricing, and a fear of **counterparty risk**: there was no guarantee that the person on the other side of a trade would actually deliver the stock or the cash.

By formalizing the rules, the Buttonwood signers helped establish trust and confidence in the financial system. In many ways, the agreement functioned as an early whitelist, where brokers served as transparent validator nodes in a distributed network. That newfound confidence proved crucial for growth: it enabled more efficient price discovery, increased liquidity, and attracted broader participation from investors. The market expanded in lockstep with the American economy as more companies sought to raise capital and more investors looked for opportunity.

In the early days, investors did not directly communicate with the exchange. Instead, they relied on brokerage offices or written instructions to place orders. On the exchange floor, brokers gathered around designated trading posts shouting bids and offers. Only when two brokers agreed on a price was the trade recorded in notebooks.

Information about prices and trades then had to be transmitted back to investors through messengers, couriers, or telegraph updates. This process introduced unavoidable delays between activity on the exchange floor and the information available to distant investors.

In effect, the market had encountered its first **latency arbitrage** problem:

participants with faster access to information could exploit those with slower access. An investor located in Boston, for example, might have to wait hours or even days for a courier to deliver trade information from New York, while a trader on the floor could act on that information immediately. The manual communication infrastructure that supported early trading had been designed for a much smaller market, and it struggled to keep pace with the accelerating volume of transactions. This latency gap meant an investor might attempt to buy a stock at a certain price, but by the time their order reached the exchange, the price could already have changed, resulting in a missed opportunity or a less favorable trade.

To address this gap, Edward A. Calahan invented the ticker tape machine, which transmitted stock prices over existing telegraph infrastructure. The device was one of the earliest examples of *near-real-time data dissemination*. Ticker tape machines functioned as specialized telegraphs whose sole purpose was to print stock symbols, prices, and volume onto a narrow paper strip. By 1929, ticker tape machines were installed in brokerage offices, investor's homes, and financial institutions across the United States, providing a continuous stream of market data to investors and traders. The result was a dramatic improvement in the speed and accuracy of information flow from Wall Street to the rest of the country. For the first time, investors could make decisions based on near-real-time data, rather than relying on delayed reports. In many ways, the ticker tape transformed financial markets from a localized trading floor into a distributed information network. Ticker tape was not without its physical limitations. Machines could only print a limited number of characters per second, and the paper strips were prone to jamming and tearing. Under normal conditions these issues were manageable, and by 1929 improved models could relay data at 285 characters per minute. But the system had a hard ceiling: when trading volume outpaced print speed, the tape fell behind, and every investor reading it was making decisions on stale prices.

From a systems perspective, the stock market had become an information network whose demand for data transmission was beginning to exceed its available bandwidth. October 1929 proved the point catastrophically. As trading volume surged past what the ticker infrastructure could handle, reporting delays grew from minutes to hours, and a feedback loop of panic selling and stale data amplified the crash far beyond what the underlying economic conditions alone would have caused.

CASE HISTORY The 1929 Crash as an Information-System Failure

Leading up to October 1929, several macroeconomic and political factors were already placing pressure on the financial system. In August 1929, the Federal Reserve raised interest rates from 5% to 6%, tightening credit conditions across the economy. Financial forecaster Roger Babson had publicly warned of a coming collapse, an event that later became known as the "Babson Break." At the same time, international instability, including the Hatry Crisis in the United Kingdom, caused British investors to withdraw capital from American markets. Domestically, income inequality meant that a relatively small portion of the population controlled most investment capital, limiting the pool of new buyers needed to sustain rising prices. Debates surrounding the proposed Smoot–Hawley Tariff Act also created fears of a global trade conflict.

These factors increased market volatility and trading activity, pushing the ticker tape system toward its operational limits.

During the week of the crash, trading volume surged to unprecedented levels. On October 21, 1929, approximately six million shares were traded, overwhelming ticker systems and causing reporting delays of nearly 100 minutes. By October 24, later known as *Black Thursday*, trading volume reached 12.9 million shares, and ticker delays expanded to nearly four hours.

For investors relying on ticker tape updates, the information they were receiving was no longer current market data, but rather a delayed record of trades that had occurred hours earlier. Investors observing falling prices on the ticker believed the market was collapsing in real time, even though the prices they were seeing reflected conditions from much earlier in the trading session. As panic selling accelerated, the volume of trades increased even further, widening the gap between actual market activity and the data being disseminated to the public.

By October 28 (*Black Monday*), trading volume reached 9.2 million shares with ticker delays approaching three hours. The following day, October 29, 1929, would become known as *Black Tuesday*: 16.4 million shares were traded, an unprecedented level of activity for the era.

The infrastructure responsible for distributing price data could no longer keep pace with the rate at which new trades were occurring. What began as an economic correction quickly escalated into a systemic information failure, where millions of investors were making decisions based on data that was already hours out of date.

Viewed through a modern lens, the events of October 1929 illustrate a critical

principle: price discovery is fundamentally constrained by the speed at which information can propagate through the system. The crash did not just expose a limitation of ticker tape; it revealed that any market whose data infrastructure cannot keep pace with its trading activity is structurally fragile. Solving this problem would require more than faster printers. It would require a fundamentally different approach to how prices were generated, matched, and distributed.

1.1.2 The Digital Revolution and the NASDAQ Era

The answer, when it finally arrived, was electronic execution. NASDAQ launched in 1971 as the first fully electronic stock market, replacing the physical ticker with digital price quotation and, eventually, automated order matching.

Where ticker tape had been a one-way broadcast—prices printed onto paper and shipped outward—electronic systems closed the loop. Quotes, orders, and confirmations could travel the same network in both directions, collapsing the gap between observing a price and acting on it. Over time, electronic routing, matching, and reporting became normal across venues. The same information bottleneck that had amplified the 1929 crash was, in principle, eliminated: digital systems could scale throughput in ways that mechanical printers never could.

Digital systems also made market data more machine-readable and more available at scale, which is exactly what quantitative research needed. But computerization did not magically "clean" the data. It introduced new forms of noise, latency effects, and cross-venue inconsistency that quants still have to model explicitly.

1.1.3 The Market as a Real-Time Decision Field

In this course, we treat the market as a high-dimensional, real-time decision environment.

Every tick is a decision event. It may be generated by a human, an automated strategy, or a hybrid workflow. Either way, each print is an action taken under constraints, not a timeless truth.

1.1.4 Primary vs Secondary Markets

Primary markets are where new securities are issued (for example, IPO allocation and initial sale). Secondary markets are where existing claims are continuously repriced and transferred.

Most daily equity trading you see is secondary-market activity. That flow is less about raising new capital for firms and more about ongoing price discovery among holders with competing views.

1.1.5 Core Market Participants

The high-level player map from lecture is the right starting point:

- Dealers/market makers: provide liquidity, quote two-sided prices, and manage inventory risk.
- Brokers: route and execute customer orders, with explicit obligations around execution quality.
- Speculators (including retail, quants, and students): seek returns from directional, statistical, or structural edges.

If you only look at exchanges and ignore these actors, you are not modeling the market. You are modeling a dashboard.

1.1.6 Dealers and Market Making

Dealers do not behave like long-only investors. Their core business is facilitating flow while managing inventory and spread risk.

In practice, many realized fills are the product of a dealer's willingness to hold risk briefly in exchange for spread economics and flow scale.

1.1.7 Brokers and Order Routing

Brokers are the transport layer between client intent and executable flow. Route choice, venue selection, and timing directly affect realized outcomes.

"Best execution" is not a slogan. It is an operational constraint with measurable consequences for slippage, fill quality, and strategy PnL.

1.1.8 Financial Intermediaries

Beyond brokers and dealers, large intermediaries (banks, funds, and asset managers) pool capital, transform exposures, and repackage risk.

These institutions change both the shape of order flow and the risk topology of the system. Ignoring them leads to naive causal stories.

1.2 System Lens

Anchor each implementation step to market mechanics first.

Chapter 2

From Intuition to Measurement

This placeholder chapter is ready for the next draft.

2.1 What We Measure

Define features only after the objective and risk assumptions are explicit.

Chapter 3

Machine Learning

From Netflix recommendations to self-driving cars, to the stock market, machine learning (ML) is quietly shaping the systems we use every day. But what is it, really?

At its heart, machine learning is about teaching computers to learn from experience. In a standard setting, this is like teaching a child to recognize a cat by showing them lots of pictures and giving feedback. Instead of writing a long, rigid list of rules (like “if it has pointy ears and whiskers, it’s a cat”), we let the computer discover the patterns in the data and improve its own performance over time.

Today, ML powers everything from voice assistants to medical diagnosis. In finance, it helps spot fraud, optimize portfolios, and predict future market direction. But financial machine learning has a unique challenge: unlike a clear picture of a cat, market data is incredibly noisy.

That is why the secret to algorithmic trading isn’t just building a complex model; it’s getting the data right to begin with. As you saw in the last chapter, high-quality, stationary data that preserves market memory is the real fuel for any financial ML system.

In this chapter, we’ll focus on the most practical and widely used branch for our pipeline: supervised learning. We’ll also briefly introduce unsupervised and reinforcement learning, so you see the big picture. You’ll learn what makes each approach unique, how algorithms use "decision trees" to mimic human logic, and how they fit into the system you’ve already started building. By the end, you’ll be ready to judge whether your model is actually uncovering a true market signal, or just memorizing historical flukes.

3.1 Principles of Machine Learning

Before we dive into the mechanics of machine learning, let us pause and consider how algorithms already shape our daily lives. Every time you unlock your phone with your face, get a movie suggestion from Netflix, or follow a recipe to bake cookies, you are relying on algorithms at work. These are not just abstract computer concepts, they are the invisible instructions guiding everything from your morning routine to the apps you use. In fact, we have all been using algorithms our entire lives, often without realizing it. Whether you are tying your shoes, solving a Rubik's cube, or deciding what to eat for breakfast, your daily routines are full of these stepwise procedures. So, what exactly is an **algorithm**? Think of it as a recipe: a clear, step-by-step set of instructions for turning raw ingredients (your input) into a finished dish (your output). In machine learning, algorithms are the backbone: they tell the computer exactly how to turn real-world data into useful predictions or decisions.

In traditional programming, a developer writes explicit rules that the computer must follow, a series of `if-then-else` statements and conditions that form the basis of a program. Want to detect spam email? A traditional programmer might write: "*if the subject line contains the word 'FREE' and the sender is unknown, mark as spam.*" This works reasonably well, until the spammers start writing "FR$$" instead of "FREE," and the programmer has to update the rules again. The fundamental limitation is clear: *the programmer must anticipate every case in advance.*

Machine learning (ML) flips this model on its head. Rather than a human writing the rules, we give the computer *examples* of correct behavior and let the computer *figure out the rules itself.* Feed it thousands of spam emails and thousands of legitimate ones, and the algorithm will discover, on its own, what patterns distinguish the two! This shift, from rule-writing to example-showing, is the core idea that makes machine learning so powerful and so different from everything that came before it (see Figure 3.1). In traditional programming, data and rules go *in*, and answers come *out.* In machine learning, data and answers (examples) go *in*, and *rules* come out. The output of a machine learning system is not a prediction by itself. In this context, a model is a compact, learned representation of the patterns in the data that can then be applied to new, unseen inputs.

Given how central machine learning has become, it's natural to wonder: if the field has existed since the 1950s, why are we only now seeing it become so widespread? The practical answer lies in two major limitations that held the field back for decades: the availability of large, high-quality datasets, and the

sheer amount of computing power required. Machine learning is, at its core, a way to overcome a lack of explicit knowledge by leveraging an abundance of data. For much of its history, this abundance simply did not exist. The advent of cloud computing and advances in computer technology have enabled researchers and companies not only to store and process vast amounts of data, but also to access it seamlessly from anywhere in the world over the internet. Seemingly, companies across the world had a collective awakening to the importance of data collection and information modeling. This shift was not just about better algorithms, but about the sudden availability of massive datasets and the computing power, especially GPUs, needed to process them [1, 3]. To put this explosion in perspective: in 2010, the world generated about 2 zettabytes of data over the entire year. [4, 3]. Today, we generate roughly 402 million terabytes of data every single day [4]. For scale, one zettabyte is equal to one billion terabytes [2]. Data collection has become so desirable that companies are even offering hardware products for free, simply to gather consumer usage data. This relentless pursuit of information underscores how valuable data has become in the modern economy, fueling advances in machine learning and finance alike.

For decades, computing was all about the Central Processing Unit (CPU). The CPU was the "chef" in the kitchen, following recipes (programs) step by step, one dish at a time. Programs were written, compiled or interpreted, and executed via the CPU, which excelled at handling a wide variety of tasks, but only a few at once. Incremental improvements in CPU performance have mostly come from smarter designs—such as adding more cores (moving from single-core to quad-core and beyond), refining how instructions are processed, and squeezing out more parallelism where possible. But even with these advances, CPUs are still generalists: great at juggling many different tasks, but not built for the kind of massive, simultaneous number-crunching that modern machine learning demands. Researchers

For the last twenty years, it was widely claimed by the media, politicians, and parents, that the video game industry wouldn't amount to much. Ironic, isn't it? The same industry, once dismissed as a frivolous pastime, became the unlikely hero in the story of machine learning's rise. Why? Because the relentless quest for ever more realistic graphics, such as lifelike shadows, explosions, and shimmering water, drove the development of the Graphical Processing Unit (GPU). Rendering a single video game scene means calculating the color and brightness of millions of pixels, all at once. Each calculation is simple, but there are so many of them that only a processor designed for massive parallelism can keep up. This native design, centered around parallel

processing, is perfectly suited for machine learning. Training a modern ML model rarely requires solving one giant, complicated equation. Instead, it involves doing millions of simple math operations, like matrix multiplications, at the same time. If a CPU is like one brilliant professor, a GPU is a stadium filled with 10,000 diligent students. Individually, each student can only do basic arithmetic, but working together, they can crunch through mountains of data.

When researchers realized they could repurpose gaming GPUs to train machine learning models, the field exploded. Algorithms that used to take months to train on a traditional computer could suddenly be completed in a few hours. Today, with the rise of cloud computing, this barrier has been significantly lowered. You no longer need to buy a supercomputer to run advanced financial models.

This democratization of cheap, powerful computation has opened the door for financial machine learning, allowing even small teams to build and test sophisticated trading algorithms that once required the resources of Wall Street giants.

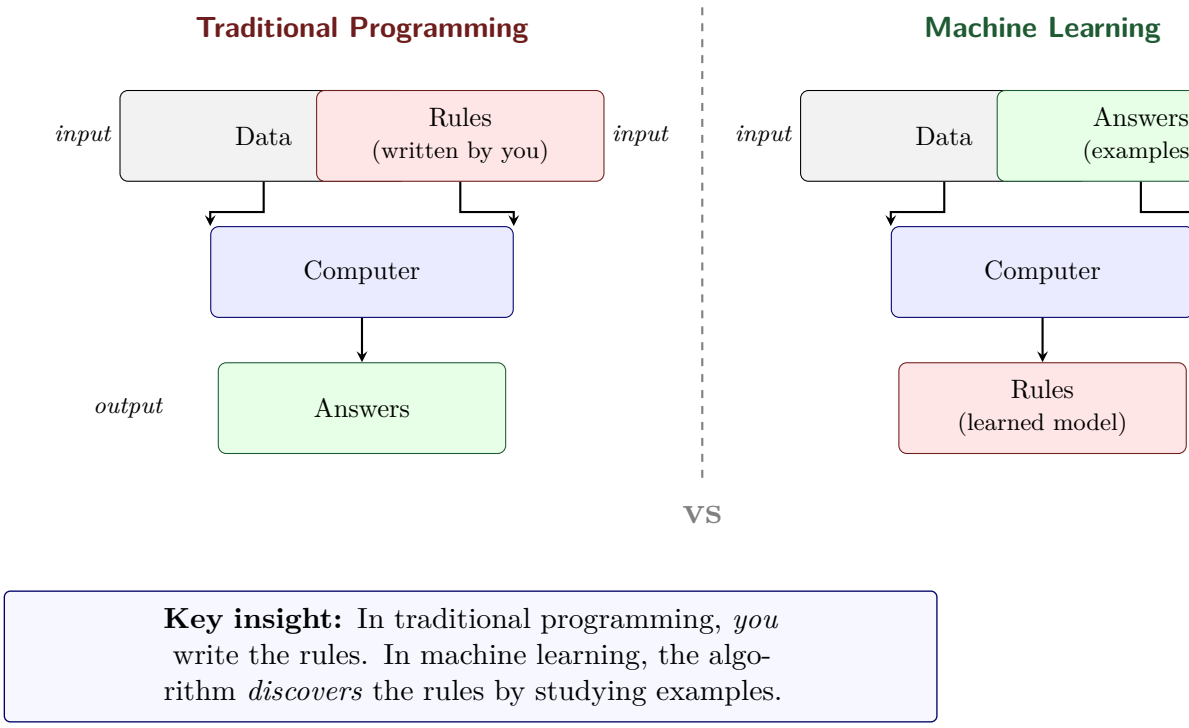

Figure 3.1: Traditional programming versus machine learning. On the left, the programmer supplies both data and hand-coded rules; the computer produces answers. On the right, the programmer supplies data and example answers; the computer produces the rules (a trained model).

3.2 Supervised Learning

Historically, epigraphy (the study of ancient inscriptions) was a painstakingly manual process, requiring years of linguistic expertise and comparison with known texts like the Rosetta Stone. The process involved meticulous analysis of symbols, their contexts, and their relationships to one another. Researchers would document each symbol, cross-reference it with known languages, and gradually piece together the meanings. In 2017, Ubisoft, a video game company, launched a research project called *The Hieroglyphics Initiative* to coincide with the release of their game Assassin's Creed Origins (which is set in ancient Egypt). Their goal was to see if machine learning could streamline the cataloging of the pharaohs' language. They did this by training a machine learning model on a large set of examples, like a stack of flashcards with both the hieroglyph and its translation. While it took centuries for scholars to build the catalog of known Egyptian symbols, a supervised model can process thousands of images in seconds, flagging uncertain symbols for human review and drastically reducing the manual labor of epigraphy. The model performed remarkably well, accurately identifying the majority of hieroglyphs and demonstrating that AI could significantly accelerate the study and translation of ancient texts while still relying on human expertise for ambiguous cases.

This application of machine learning is known as **supervised learning**, where the model is trained on a labeled dataset, learning to map inputs (like images of hieroglyphs) to outputs (their translations). This pattern, learning from examples with known answers, underpins everything from image recognition to predicting tomorrow's stock price.

Supervised learning is a type of machine learning where the model is trained on a **labeled dataset**, meaning that each input data point is paired with the correct output. The model learns to map inputs to outputs by finding patterns in the data. Once trained, the model can make predictions on new, unseen data. This labeled dataset is broken into two key pairs: **features** and **labels**. Features are the measurable properties or characteristics of the data, while labels are the outcomes or targets that we want to predict.

In our financial context, features might include recent price changes, trading volume, or technical indicators such as the relative strength index or exponential moving average. Labels are the outcomes we want to predict—like whether a trade will be profitable or not.

For many years, researchers tried to predict the exact next price of an asset. But as we've discussed in previous chapters, so many unpredictable factors influence price that this is a daunting task. Instead, we often simplify the

problem: rather than guessing the next price, we ask the model to classify the next movement as "profitable" or "unprofitable" for a given trading strategy.

This leads to two main types of supervised learning problems:

- **Classification:** The model assigns each input to a category, such as "profitable" or "unprofitable." For example, using a moving average crossover strategy, the model learns from historical data to predict whether the next trade will succeed or not.
- **Regression:** The model predicts a continuous value, like the exact next price of an asset.

In this book, we'll focus mainly on classification, since it's often more practical and robust in financial applications. In traditional programming, we know a developer is responsible for the rules, but we noted that ML flips this model on its head. So, how does a model actually learn from the data? If anyone remembers a game called "'20 Questions"' you can relate to how a model learns. In this game, one player thinks of an object, and the other player asks yes/no questions to narrow down the possibilities. In a similar way, a machine learning model learns by asking questions about the features in the data. Similar to the different strategies a player might use in 20 Questions, there are different algorithms that can be used to learn from a dataset, each with its own strengths and weaknesses.

- **transformers/CNN** transformers and convolutional neural networks (CNNs) are powerful for processing sequential or spatial data, such as time series or images.
- **decision trees** decision trees are a simple yet effective algorithm that splits the data into branches based on feature values, making decisions at each node.
- **random forests** random forests are an ensemble method that combines multiple decision trees to improve accuracy and reduce overfitting.
- **gradient boosting** gradient boosting is another ensemble technique that builds trees sequentially, where each new tree corrects the errors of the previous ones.

In the next section, we'll explore how decision trees work, and why they are a popular choice for financial machine learning applications. We'll also discuss how to evaluate the performance of these models and ensure they generalize well to new data, rather than just memorizing historical patterns.

3.3 Random Forest

Have you ever played 20 Questions with a friend? If so, you've already used a decision tree!

In our previous section, we introduced multiple supervised learning algorithms, including decision trees and ensemble methods like random forests. A decision tree is just a hierarchical structure (top-down) that splits the data into branches based on feature values, making decisions at each node. A model reaches a decision by traversing a decision tree from the **root node** to a **leaf node**, where the final prediction is made.

The root node is the topmost node in a decision tree, representing the entire dataset. A leaf node, on the other hand, is a terminal node that provides the final output or prediction for a given input. It's a node without any child nodes (branches), and it represents the end of a decision path in the tree.

Along the way, the model passes through internal nodes. Think of these as the decision points where you ask, "Does it live in water?" or "Is it man-made?"

In the example visualized in Figure **??**, a 5-level decision tree is used to play the game of 20 Questions. The root node in this case is *Is it a living thing?*. There are several internal nodes such as *Does it live in water?*, *Is it an animal?*, and *Is it man-made?*. The leaf nodes represent the final answers, such as *whale*, *tree*, or *rock*.

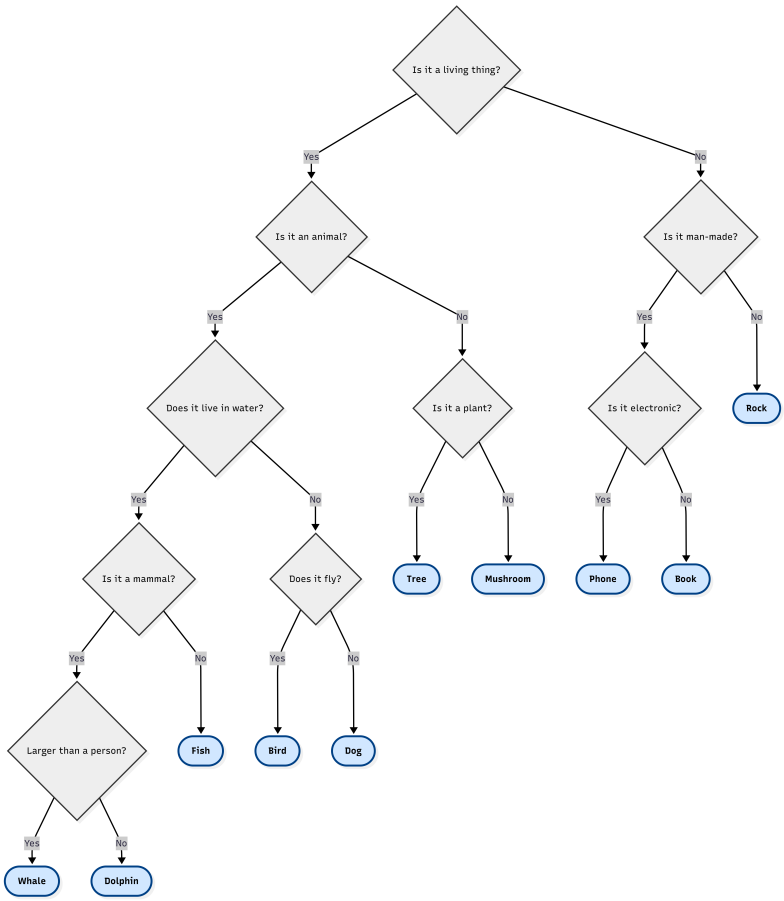

Figure 3.2: A 5-level decision tree for 20 Questions.

Bibliography

[1] IBM. Bringing big data to the enterprise, 2012. Accessed: 2026-03-10.

[2] National Institute of Standards and Technology. Prefixes for binary multiples, 2024. Accessed: 2026-03-10.

[3] David Reinsel, John Gantz, and John Rydning. Worldwide global datasphere forecast, 2021–2025. Technical report, IDC, 2021. Accessed: 2026-03-10.

[4] Statista. Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2025, 2024. Accessed: 2026-03-10.